# 211-Discussion

Yu-Shiuan (Lily) Huang

Fall 2023

**Discussion I**

For today's discussion, we'll first introduce **LaTeX** and **R Markdown**, which are two very useful typesetting systems for you to generate neat documents for your problem sets and research projects. The second part of discussion will review some basic R skills when exploring a new dataset.

**1. LaTeX** LaTeX is a high-quality typesetting system; it includes features designed for the production of technical and scientific documentation. LaTeX is the de facto standard for the communication and publication of scientific documents.

- Installation: LaTeX installation guide
- Short Tutorials: LaTeX tutorial from Gov department PreFresher

If you prefer not to download the LaTeX compiler and editor to your laptop, you can use Overleaf, which is an online LaTeX editor that is very easy to use and work on projects with multiple collaborators. It also provides many templates for you to choose from. Here is a brief introduction video of how to use Overleaf.

- To create a nice table output in Overleaf, TablesGenerator is very helpful. You can also use `stargazer` in R to create regression tables in LaTeX form and copy them into Overleaf. Here is a handout of how to use the `stargazer` package (we will talk more on this in the future).
- To insert math equations in Overleaf, LaTeX Equation Editor would be your go-to.

Here is an Overleaf template for POL 211 Problem Set I. Please feel free to copy it to your future work!

**2. R Markdown** Another way to submit both your answers and R code with only one pdf document is by using **R Markdown**.

R Markdown is a markup language that allows you to create dynamic documents in R programming language. With R Markdown, you can easily combine text, code, and visualizations in a single document. R Markdown documents can be rendered in a variety of output formats, including HTML, PDF, Microsoft Word, and more.

R Markdown provides a way to weave together narrative text, code, and results into a single document that can be easily shared and reproduced. It allows you to write code and see the results of that code inline, making it easy to explain complex statistical analyses or data visualizations.

Here is a thorough introduction of how to use R Markdown.

For the following topic on exploring a new dataset using R, all the materials will be shown in R Markdown format. You are also more than welcome to upload your problem set:

- by knitting your R Markdown to **html** first, then open it in a browser, and then save it as a pdf.(Just click the `Knit` button!)

- by knitting your R Markdown to **pdf** directly if you have already downloaded a LaTeX compiler in your computer.

  When knitting to pdf, all of the contents in R Markdown will be presented in LaTeX format automatically. If you don't have a LaTeX compiler installed on your machine, you will encounter an error message when attempting to knit your R Markdown to PDF. If you need more help, here is a guide I found online that provides step-by-step instructions on how to do it.

- A R Markdown template for Problem Set I can be downloaded from Canvas-Files.

**3. Exploring a New Dataset using R (Part I)** Note: All these R codes are prepared by Prof. Ryan Hubert.

**a. Load R packages** There are many useful packages in R that can be installed freely. If it is the first time you are going to use a package, here are the steps:

- Install it
- library it (which means telling R you are going to use it)

Once you have installed the package, you don't have to install it every time you open R studio. However, you still have to load it through the `library` function every time.

```
# Install packages install.packages('tidyverse') #
# manipulate data install.packages('Matrix') # calculate
# rank these are the packages we will use for today

# load packages
library(tidyverse)
library(Matrix)
```

The `tidyverse` is a powerful collection of R packages that are actually data tools for transforming and visualizing data. All packages of the `tidyverse` share an underlying philosophy and common APls.

The core packages within the `tidyverse` are:

- `ggplot2`, which implements the grammar of graphics. You can use it to visualize your data.
- `dplyr` is a grammar of data. You can use it to solve the most common data manipulation challenges.
- `tidyr` helps you to create tidy data or data where each variable is in a column, each observation is a row end each value is a column, each observation is a row end each value is a cell.
- `readr` is a fast and friendly way to read rectangular
- `purrr` enhances R's functional programming (FP) toolkit by providing a complete and consistent set of tools for working with functions and vectors.
- `tibble` is a modern re-imaginging of the data
- `stringr` provides a cohesive set of functions designed to make working with strings as easy as possible
- `forcats` provide a suite of useful tools that solve common problems with factors.

Highly recommend this data wrangling cheat sheet with `dplyr` and `tidyr` in R!

**b. Import data** For today, we are using `USDC-DATASET-JOP.csv`. You can download this csv file from Canvas or from here.

```
# Import data
df <- read_csv("/Users/yu-shiuanhuang/Desktop/method-sequence/data/USDC-DATASET-JOP.csv")
# Here we assign the dataset as an object called "df"

## Note: you create new objects by assigning them with the
## `<-` phrase. You can also use an equal sign `=`.
```

Here are some tips for how to find the file path of the data you are going to use.

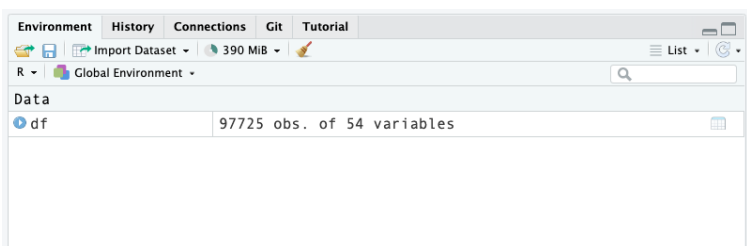
- First, where did you store your data? In which folder? In my case, I stored the data in the `data` folder within the `method-sequence` folder.
- Second, use `getwd` function to get the grasp of how does a file path look like. `getwd` function returns an absolute file path representing the current working directory of the R process.

```
getwd()
```

```
## [1] "/Users/yu-shiuanhuang/Desktop/method-sequence/docs"
```

As you can see, my current working directory is `"/Users/yu-shiuanhuang/Desktop/method-sequence/docs"`. Since I know that I stored the data in the `data` folder within the `method-sequence` folder, I can simply just change the file path to `"/Users/yu-shiuanhuang/Desktop/method-sequence/data/USDC-DATASET-JOP.csv"` and put it in the `read_csv` function so R knows where to read the file you want.

If you import the data successfully, you should be able to see df popping up in the environment section.




Sometimes the dataset we are interested in may be stored in a different form, such as `dta`, `spss`, `xlsx`, `rds`, `rdata`, etc. You can find the corresponded codes from here.

**c. Basic information about the dataset** What is the size of the dataset?

```
# the number of row is:
nrow(df)
```

```
## [1] 97725
```

How many observations (rows) and variables (columns) we have in the dataset?

```
# the number of column is:
ncol(df)
```

```
## [1] 54
```

You can also use `dim` function to get the size of the dataset.

```
dim(df)
```

```
## [1] 97725    54
```

```r
## this function gives you the number of rows and columns
## at the same time
```

The size of this dataset is 97,725 x 54.

What does the dataset actually look like?

If you want to just quickly see the first six rows to get a feel for what the dataset looks like:

```r
head(df)
```

```
## # A tibble: 6 x 54
##   file      OUTCOME jrepublican jpresident jid_anon jyears APPEAL judge_replaced
##   <chr>     <chr>         <dbl> <chr>         <dbl>  <dbl>  <dbl>          <dbl>
## 1 /dockets~ jud_de~           1 Reagan        90059     13      0              0
## 2 /dockets~ dism_o~           1 Reagan        90048      9      0              0
## 3 /dockets~ dism_p~           1 Reagan        90012     11      0              0
## 4 /dockets~ dism_o~           1 Bush41        90016      3      0              0
## 5 /dockets~ dism_v~           1 Reagan        90096     10      0              0
## 6 /dockets~ dism_v~           1 Reagan        90158     11      0              0
## # i 46 more variables: jsenior <dbl>, court <chr>, division <dbl>, YEAR <dbl>,
## #   QUARTER <chr>, nature_of_suit <dbl>, SECTION <chr>, ORIGIN <dbl>,
## #   JURIS <chr>, jury_demand <chr>, PROSE <dbl>, COUNTY <chr>, pla_count <dbl>,
## #   pla_count_prose <dbl>, pla_count_anonymous <dbl>, pla_count_IND <dbl>,
## #   pla_count_BUS <dbl>, pla_count_GOV <dbl>, pla_count_LAW <dbl>,
## #   pla_count_OTH <dbl>, pla_count_repeat <dbl>, pla_acount <dbl>,
## #   pla_acount_repeat <dbl>, def_count <dbl>, def_count_prose <dbl>, ...
```

Note that a lot of the variables are not visible because R will try to clean up output.

One convenient thing is that you can see the type of each variable.Here is the full list of all the possible types of variables (in tidyverse).

If you want to look at the whole dataset, use `View(df)` command and it will open in a new tab in R.

**d. Exploring the variables** Most people prefer to look at variables using their names.

How to ask R to tell us the name of each variables?

```r
# Name of each column
colnames(df)
```

```
##  [1] "file"                "OUTCOME"             "jrepublican"
##  [4] "jpresident"          "jid_anon"            "jyears"
##  [7] "APPEAL"              "judge_replaced"      "jsenior"
## [10] "court"               "division"            "YEAR"
## [13] "QUARTER"             "nature_of_suit"      "SECTION"
## [16] "ORIGIN"              "JURIS"               "jury_demand"
## [19] "PROSE"               "COUNTY"              "pla_count"
## [22] "pla_count_prose"     "pla_count_anonymous" "pla_count_IND"
## [25] "pla_count_BUS"       "pla_count_GOV"       "pla_count_LAW"
## [28] "pla_count_OTH"       "pla_count_repeat"    "pla_acount"
## [31] "pla_acount_repeat"   "def_count"           "def_count_prose"
## [34] "def_count_anonymous" "def_count_IND"       "def_count_BUS"
## [37] "def_count_GOV"       "def_count_LAW"       "def_count_OTH"
## [40] "def_count_repeat"    "def_acount"          "def_acount_repeat"
## [43] "oth_count"           "oth_count_prose"     "oth_count_anonymous"
## [46] "oth_count_IND"       "oth_count_BUS"       "oth_count_GOV"
## [49] "oth_count_LAW"       "oth_count_OTH"       "oth_count_repeat"
```

```
## [52] "oth_acount"          "oth_acount_repeat"   "ifp_denial"
```

How to call out one specific variable we are interested in from the dataset? There are multiple ways to look at an arbitrary variable:

```
## Use column number
df[, 2]
df[[2]]

## Use variable name
df[, "OUTCOME"]
df[["OUTCOME"]]
df$OUTCOME  ## This is the most common way to do it
```

```
head(df$OUTCOME)  ## To see the first six rows of this variable
```

```
## [1] "jud_default_plaintiff" "dism_other"            "dism_pros"
## [4] "dism_other"            "dism_vol"              "dism_vol"
```

What kind of variable is the `OUTCOME` variable? It looks like a categorical variable!

The next question would be what are the categories (called "levels" in R) of this variable?

```
# This asks R to give us a list of all the values used in
# this variable.
unique(df$OUTCOME)
```

```
##  [1] "jud_default_plaintiff"  "dism_other"             "dism_pros"
##  [4] "dism_vol"               "settlement"             "jud_motion_defendant"
##  [7] "dism_juris"             "remand"                 "jud_misc_NA"
## [10] "jud_misc_both"          "jud_motion_plaintiff"   "jud_misc_defendant"
## [13] "jud_jury_defendant"     "transfer"               "jud_bench_defendant"
## [16] "jud_misc_plaintiff"     "jud_consent_plaintiff"  "jud_jury_plaintiff"
## [19] "jud_default_both"       "jud_motion_both"        "jud_default_defendant"
## [22] "jud_bench_plaintiff"    "jud_jury_both"          "jud_consent_both"
## [25] "jud_bench_both"         "jud_directed_defendant" "jud_consent_defendant"
## [28] "jud_motion_NA"          "jud_jury_NA"            "jud_bench_NA"
## [31] "jud_consent_NA"         "jud_directed_plaintiff" "jud_default_NA"
## [34] "jud_directed_NA"
```

There are 34 categories in this variable.

Here's an issue: when R loaded the dataset, it didn't code this variable as a categorical variable. Since the `OUTCOME` variable is coded as strings/texts in the dataset, in R, the type of this variable is character.

```
# Look at what this says about the variable type:
type_sum(df$OUTCOME)
```

```
## [1] "chr"
```

```
# When you summarize it in R, R doesn't provide you with
# meaningful information
summary(df$OUTCOME)
```

```
##    Length     Class      Mode
##     97725 character character
```

Good news is that we can transform the type of a variable in R.

```
# lets's change the type to a categorical variable, known
# as a 'factor' variable in R:
```

```r
df$OUTCOME <- as.factor(df$OUTCOME)
## In order to override the original variable, we have to
## assign the new version of the variable to the original
## one.

# After factorizing, R can summarize in a more meaningful
# way: what is the number of each category?
summary(df$OUTCOME)
```

```
##             dism_juris             dism_other              dism_pros
##                   1321                  16198                   3123
##               dism_vol         jud_bench_both    jud_bench_defendant
##                  17805                     10                    192
##           jud_bench_NA    jud_bench_plaintiff       jud_consent_both
##                     22                    110                    199
##  jud_consent_defendant         jud_consent_NA  jud_consent_plaintiff
##                     48                     93                    391
##       jud_default_both  jud_default_defendant         jud_default_NA
##                      5                     69                      7
##  jud_default_plaintiff jud_directed_defendant        jud_directed_NA
##                    228                     34                      1
## jud_directed_plaintiff          jud_jury_both     jud_jury_defendant
##                      6                     73                   1260
##            jud_jury_NA     jud_jury_plaintiff          jud_misc_both
##                    121                    528                    399
##     jud_misc_defendant            jud_misc_NA     jud_misc_plaintiff
##                   1355                   2239                    413
##        jud_motion_both   jud_motion_defendant          jud_motion_NA
##                    428                  10510                    821
##   jud_motion_plaintiff                 remand             settlement
##                    629                   4546                  32627
##               transfer
##                   1914
```

**e. Exploring the observations** Each row (observation) in this dataset is a court case. We can also use R to look at an arbitrary one.

```r
df[5, ]  # This means that we want to call out the fifth row of the data.
```

```
## # A tibble: 1 x 54
##   file      OUTCOME jrepublican jpresident jid_anon jyears APPEAL judge_replaced
##   <chr>     <fct>         <dbl> <chr>         <dbl>  <dbl>  <dbl>          <dbl>
## 1 /dockets~ dism_v~           1 Reagan        90096     10      0              0
## # i 46 more variables: jsenior <dbl>, court <chr>, division <dbl>, YEAR <dbl>,
## #   QUARTER <chr>, nature_of_suit <dbl>, SECTION <chr>, ORIGIN <dbl>,
## #   JURIS <chr>, jury_demand <chr>, PROSE <dbl>, COUNTY <chr>, pla_count <dbl>,
## #   pla_count_prose <dbl>, pla_count_anonymous <dbl>, pla_count_IND <dbl>,
## #   pla_count_BUS <dbl>, pla_count_GOV <dbl>, pla_count_LAW <dbl>,
## #   pla_count_OTH <dbl>, pla_count_repeat <dbl>, pla_acount <dbl>,
## #   pla_acount_repeat <dbl>, def_count <dbl>, def_count_prose <dbl>, ...
```

```r
# If you want to see all the variables for that
# observation, here are some options:
t(df[5, ])  ## downside: this treats all the values like strings!
```

```
##                      [,1]
## file                 "/dockets-cacd/1995/199501-00068.html"
## OUTCOME              "dism_vol"
## jrepublican          "1"
## jpresident           "Reagan"
## jid_anon             "90096"
## jyears               "10"
## APPEAL               "0"
## judge_replaced       "0"
## jsenior              "0"
## court                "cacd"
## division             "2"
## YEAR                 "1995"
## QUARTER              "1995Q1"
## nature_of_suit       "442"
## SECTION              "1983"
## ORIGIN               "1"
## JURIS                "federal_question"
## jury_demand          "Plaintiff"
## PROSE                NA
## COUNTY               "06037"
## pla_count            "1"
## pla_count_prose      "0"
## pla_count_anonymous "0"
## pla_count_IND        "0.7538"
## pla_count_BUS        "0.1453"
## pla_count_GOV        "0.0485"
## pla_count_LAW        "0.0315"
## pla_count_OTH        "0.021"
## pla_count_repeat     "0"
## pla_acount           "1"
## pla_acount_repeat    "1"
## def_count            "4"
## def_count_prose      "0"
## def_count_anonymous "1"
## def_count_IND        "2.7986"
## def_count_BUS        "0.7653"
## def_count_GOV        "0.2055"
## def_count_LAW        "0.1402"
## def_count_OTH        "0.0903"
## def_count_repeat     "1"
## def_acount           "0"
## def_acount_repeat    "0"
## oth_count            "0"
## oth_count_prose      "0"
## oth_count_anonymous "0"
## oth_count_IND        "0"
## oth_count_BUS        "0"
## oth_count_GOV        "0"
## oth_count_LAW        "0"
## oth_count_OTH        "0"
## oth_count_repeat     "0"
## oth_acount           "0"
## oth_acount_repeat    "0"
```

```
## ifp_denial          "0"
```

```
as.data.frame(df[5, ])  ## downside: sort of ugly and hard to see
```

```
##                                  file  OUTCOME jrepublican jpresident jid_anon
## 1 /dockets-cacd/1995/199501-00068.html dism_vol           1     Reagan    90096
##   jyears APPEAL judge_replaced jsenior court division YEAR QUARTER
## 1     10      0              0       0  cacd        2 1995  1995Q1
##   nature_of_suit SECTION ORIGIN            JURIS jury_demand PROSE COUNTY
## 1            442    1983      1 federal_question   Plaintiff    NA  06037
##   pla_count pla_count_prose pla_count_anonymous pla_count_IND pla_count_BUS
## 1         1               0                   0        0.7538        0.1453
##   pla_count_GOV pla_count_LAW pla_count_OTH pla_count_repeat pla_acount
## 1        0.0485        0.0315         0.021                0          1
##   pla_acount_repeat def_count def_count_prose def_count_anonymous def_count_IND
## 1                 1         4               0                   1        2.7986
##   def_count_BUS def_count_GOV def_count_LAW def_count_OTH def_count_repeat
## 1        0.7653        0.2055        0.1402        0.0903                1
##   def_acount def_acount_repeat oth_count oth_count_prose oth_count_anonymous
## 1          0                 0         0               0                   0
##   oth_count_IND oth_count_BUS oth_count_GOV oth_count_LAW oth_count_OTH
## 1             0             0             0             0             0
##   oth_count_repeat oth_acount oth_acount_repeat ifp_denial
## 1                0          0                 0          0
```

```
## To draw out an element from a data, you have to specify
## a row number and a column number.
df[5, 1]
```

```
## # A tibble: 1 x 1
##   file
##   <chr>
## 1 /dockets-cacd/1995/199501-00068.html
```

Usually, you only need to see some of the variables. For example: suppose we want to know which judge heard this case (`jid_anon`), what the case outcome was (`OUTCOME`) and the party of the judge's appointing president (`jrepublican`).

```
df[5, c("jid_anon", "OUTCOME", "jrepublican")]
```

```
## # A tibble: 1 x 3
##   jid_anon OUTCOME  jrepublican
##      <dbl> <fct>          <dbl>
## 1    90096 dism_vol           1
```

A more neat way to write the code by using `tidyverse` package.

```
# using pipes to do the same thing; use select function
# from tidyverse to pick variables
df[5, ] %>%
    select(jid_anon, OUTCOME, jrepublican)
```

```
## # A tibble: 1 x 3
##   jid_anon OUTCOME  jrepublican
##      <dbl> <fct>          <dbl>
## 1    90096 dism_vol           1
```

`%>%` is called "pipes." Pipes take the output from one function and feed it to the first argument of the next function.

We can also find a row by "filtering" the data based on some criteria.

What if I want to find a case by its ID number? Eg, "/dockets-cacd/2007/200738-00195.html"

```
# use filter function from tidyverse to set the condition
df %>%
    filter(file == "/dockets-cacd/2007/200738-00195.html")
```

```
## # A tibble: 1 x 54
##   file      OUTCOME jrepublican jpresident jid_anon jyears APPEAL judge_replaced
##   <chr>     <fct>         <dbl> <chr>         <dbl>  <dbl>  <dbl>          <dbl>
## 1 /dockets~ settle~           1 Bush43        90104      0      0              0
## # i 46 more variables: jsenior <dbl>, court <chr>, division <dbl>, YEAR <dbl>,
## #   QUARTER <chr>, nature_of_suit <dbl>, SECTION <chr>, ORIGIN <dbl>,
## #   JURIS <chr>, jury_demand <chr>, PROSE <dbl>, COUNTY <chr>, pla_count <dbl>,
## #   pla_count_prose <dbl>, pla_count_anonymous <dbl>, pla_count_IND <dbl>,
## #   pla_count_BUS <dbl>, pla_count_GOV <dbl>, pla_count_LAW <dbl>,
## #   pla_count_OTH <dbl>, pla_count_repeat <dbl>, pla_acount <dbl>,
## #   pla_acount_repeat <dbl>, def_count <dbl>, def_count_prose <dbl>, ...
```

What if I want to find all cases heard by judges appointed by Bush 43?

```
df %>%
    filter(jpresident == "Bush43")
```

```
## # A tibble: 17,622 x 54
##    file     OUTCOME jrepublican jpresident jid_anon jyears APPEAL judge_replaced
##    <chr>    <fct>         <dbl> <chr>         <dbl>  <dbl>  <dbl>          <dbl>
##  1 /docket~ jud_mo~           1 Bush43        90131      1      0              0
##  2 /docket~ dism_v~           1 Bush43        90131      0      0              0
##  3 /docket~ dism_p~           1 Bush43        90131      0      0              0
##  4 /docket~ dism_j~           1 Bush43        90131      0      0              0
##  5 /docket~ dism_o~           1 Bush43        90159      0      0              0
##  6 /docket~ dism_p~           1 Bush43        90159      0      0              0
##  7 /docket~ dism_o~           1 Bush43        90159      0      0              0
##  8 /docket~ dism_o~           1 Bush43        90159      0      0              0
##  9 /docket~ dism_o~           1 Bush43        90131      0      1              0
## 10 /docket~ dism_o~           1 Bush43        90159      0      1              1
## # i 17,612 more rows
## # i 46 more variables: jsenior <dbl>, court <chr>, division <dbl>, YEAR <dbl>,
## #   QUARTER <chr>, nature_of_suit <dbl>, SECTION <chr>, ORIGIN <dbl>,
## #   JURIS <chr>, jury_demand <chr>, PROSE <dbl>, COUNTY <chr>, pla_count <dbl>,
## #   pla_count_prose <dbl>, pla_count_anonymous <dbl>, pla_count_IND <dbl>,
## #   pla_count_BUS <dbl>, pla_count_GOV <dbl>, pla_count_LAW <dbl>,
## #   pla_count_OTH <dbl>, pla_count_repeat <dbl>, pla_acount <dbl>, ...
```

The reason why we need to add quotation mark when setting the condition in the `filter` function is that both of the `file` and `jpresident` variables are coded in character type (categorical) in R. If we filter our cases based on whether the judge of the case is assigned by a Republican president (when `jrepublican` == 1), we don't have to add quotation mark in the condition as this variable is coded as the type of double (dbl) in R, which is a class of numeric.

```
df %>%
    filter(jrepublican == 1)
```

```
## # A tibble: 42,679 x 54
##    file     OUTCOME jrepublican jpresident jid_anon jyears APPEAL judge_replaced
##    <chr>    <fct>         <dbl> <chr>         <dbl>  <dbl>  <dbl>          <dbl>
```

```
##  1 /docket~ jud_de~           1 Reagan        90059     13      0              0
##  2 /docket~ dism_o~           1 Reagan        90048      9      0              0
##  3 /docket~ dism_p~           1 Reagan        90012     11      0              0
##  4 /docket~ dism_o~           1 Bush41        90016      3      0              0
##  5 /docket~ dism_v~           1 Reagan        90096     10      0              0
##  6 /docket~ dism_v~           1 Reagan        90158     11      0              0
##  7 /docket~ settle~           1 Reagan        90158     11      0              0
##  8 /docket~ settle~           1 Ford          90218     19      0              0
##  9 /docket~ jud_mo~           1 Nixon         90148     24      0              0
## 10 /docket~ dism_j~           1 Reagan        90005     13      0              0
## # i 42,669 more rows
## # i 46 more variables: jsenior <dbl>, court <chr>, division <dbl>, YEAR <dbl>,
## #   QUARTER <chr>, nature_of_suit <dbl>, SECTION <chr>, ORIGIN <dbl>,
## #   JURIS <chr>, jury_demand <chr>, PROSE <dbl>, COUNTY <chr>, pla_count <dbl>,
## #   pla_count_prose <dbl>, pla_count_anonymous <dbl>, pla_count_IND <dbl>,
## #   pla_count_BUS <dbl>, pla_count_GOV <dbl>, pla_count_LAW <dbl>,
## #   pla_count_OTH <dbl>, pla_count_repeat <dbl>, pla_acount <dbl>, ...
```

**f. Some measurement issues** What kinds of variable do we have?

There are slight differences between the way a computer stores/understands data and the way we conceptualize it. In this dataset, we have some variables on cardinal scales.

For example: How many plaintiffs are named in each case? (Why is this on a cardinal scale?)

```
head(df$pla_count)
```

```
## [1] 1 1 1 1 1 1
```

What is the data type? It's "dbl" which means it's a number that need not be an integer (can be fractional number, 3.51)

```
type_sum(df$pla_count)
```

```
## [1] "dbl"
```

Note: this variable is actually comprised of integers, we could convert it if we want, but this doesn't really matter. (Why? 11.0 = 11)

```
df$pla_count <- as.integer(df$pla_count)
type_sum(df$pla_count)
```

```
## [1] "int"
```

We also have some variables that are ordinal.

For example, in what years were cases filed? (Why is this on an ordinal scale?)

```
head(df$YEAR)
```

```
## [1] 1995 1995 1995 1995 1995 1995
```

What is the data type? It's also "dbl".

```
type_sum(df$YEAR)
```

```
## [1] "dbl"
```

Acknowledge, this is bad! We do not want R to treat this as a cardinal variable.

For example, it (incorrectly) allows us to add them:

```r
df$YEAR[1] + df$YEAR[2]
```

```
## [1] 3990
```

```r
## Notice here the change in indexing! Remember that when
## you call out one variable from the data, it is a vector.
## To draw out an element from a vector, you only have to
## specify one number inside [ ].
```

So, we should change it to a factor variable so that we don't make this mistake.

```r
df$YEAR <- as.factor(df$YEAR)
df$YEAR[1] + df$YEAR[2]
```

```
## Warning in Ops.factor(df$YEAR[1], df$YEAR[2]): '+' not meaningful for factors
```

```
## [1] NA
```

We got an error when we tried to do something silly, which is good!

In the analysis for Ryan's paper, they had to convert the `OUTCOME` variable to dummies. How can we do it in R?

Recall that there are 34 unique categories in the `OUTCOME` variable.

```r
unique(df$OUTCOME)
```

```
##  [1] jud_default_plaintiff  dism_other             dism_pros
##  [4] dism_vol               settlement             jud_motion_defendant
##  [7] dism_juris             remand                 jud_misc_NA
## [10] jud_misc_both          jud_motion_plaintiff   jud_misc_defendant
## [13] jud_jury_defendant     transfer               jud_bench_defendant
## [16] jud_misc_plaintiff     jud_consent_plaintiff  jud_jury_plaintiff
## [19] jud_default_both       jud_motion_both        jud_default_defendant
## [22] jud_bench_plaintiff    jud_jury_both          jud_consent_both
## [25] jud_bench_both         jud_directed_defendant jud_consent_defendant
## [28] jud_motion_NA          jud_jury_NA            jud_bench_NA
## [31] jud_consent_NA         jud_directed_plaintiff jud_default_NA
## [34] jud_directed_NA
## 34 Levels: dism_juris dism_other dism_pros dism_vol ... transfer
```

When we want to make each category of this variable into dummies, it means that we need to create 34 new dummy variables (34 new columns for the dataset). For example, the new dummy variable `dism_other` will be coded as 1 if the `OUTCOME` variable is coded as dism_other. Let's use `ifesle` function to create this dummy!

```r
df$dism_other <- ifelse(df$OUTCOME == "dism_other", 1, 0)
```

Let's check if we did it correctly.

```r
df %>%
    select(OUTCOME, dism_other) %>%
    head()
```

```
## # A tibble: 6 x 2
##   OUTCOME               dism_other
##   <fct>                      <dbl>
## 1 jud_default_plaintiff          0
## 2 dism_other                     1
## 3 dism_pros                      0
```

```
## 4 dism_other                      1
## 5 dism_vol                        0
## 6 dism_vol                        0
```

But there is a total of 34 categories in the `OUTCOME` variable, it would be a waste of time to generate all 34 dummies if we have to retype the above one by one for each category. Luckily, R is here to help us! We can run a `forloop` function to ask R to do the same thing for all the 34 categories.

```r
for (v in levels(df$OUTCOME)) {
    print(v)
    df[[v]] <- ifelse(df$OUTCOME == v, 1, 0)  ## df[[v]] is a way to call out/create a variable
}  ## This is called 'forloop.'
```

```
## [1] "dism_juris"
## [1] "dism_other"
## [1] "dism_pros"
## [1] "dism_vol"
## [1] "jud_bench_both"
## [1] "jud_bench_defendant"
## [1] "jud_bench_NA"
## [1] "jud_bench_plaintiff"
## [1] "jud_consent_both"
## [1] "jud_consent_defendant"
## [1] "jud_consent_NA"
## [1] "jud_consent_plaintiff"
## [1] "jud_default_both"
## [1] "jud_default_defendant"
## [1] "jud_default_NA"
## [1] "jud_default_plaintiff"
## [1] "jud_directed_defendant"
## [1] "jud_directed_NA"
## [1] "jud_directed_plaintiff"
## [1] "jud_jury_both"
## [1] "jud_jury_defendant"
## [1] "jud_jury_NA"
## [1] "jud_jury_plaintiff"
## [1] "jud_misc_both"
## [1] "jud_misc_defendant"
## [1] "jud_misc_NA"
## [1] "jud_misc_plaintiff"
## [1] "jud_motion_both"
## [1] "jud_motion_defendant"
## [1] "jud_motion_NA"
## [1] "jud_motion_plaintiff"
## [1] "remand"
## [1] "settlement"
## [1] "transfer"
```

```r
ncol(df)
```

```
## [1] 88
```

You can find that the number of columns increased to 88 from 54 after creating 34 dummies.

**g. The rank of a matrix** The rank of a dataset is the number of "linearly independent" variables.

```r
library(Matrix)  ## This package has a function for calculating the rank of a matrix

# Let's create a fake dataset
df2 <- data.frame(var1 = seq(1, 100, 3))  ## Creates 34 rows

df2$var2 <- (df2$var1/pi)  ## create a new variable called var2 in df2
pi  ## pi is an inbuilt R constant whose value is 3.141593
```

```
## [1] 3.141593
```

```r
df2$var3 <- 6
df2
```

```
##    var1       var2 var3
## 1     1  0.3183099    6
## 2     4  1.2732395    6
## 3     7  2.2281692    6
## 4    10  3.1830989    6
## 5    13  4.1380285    6
## 6    16  5.0929582    6
## 7    19  6.0478878    6
## 8    22  7.0028175    6
## 9    25  7.9577472    6
## 10   28  8.9126768    6
## 11   31  9.8676065    6
## 12   34 10.8225361    6
## 13   37 11.7774658    6
## 14   40 12.7323954    6
## 15   43 13.6873251    6
## 16   46 14.6422548    6
## 17   49 15.5971844    6
## 18   52 16.5521141    6
## 19   55 17.5070437    6
## 20   58 18.4619734    6
## 21   61 19.4169031    6
## 22   64 20.3718327    6
## 23   67 21.3267624    6
## 24   70 22.2816920    6
## 25   73 23.2366217    6
## 26   76 24.1915513    6
## 27   79 25.1464810    6
## 28   82 26.1014107    6
## 29   85 27.0563403    6
## 30   88 28.0112700    6
## 31   91 28.9661996    6
## 32   94 29.9211293    6
## 33   97 30.8760590    6
## 34  100 31.8309886    6
```

What is the rank of this matrix (the fake dataset we created)?

```r
rankMatrix(as.matrix(df2))
```

```
## [1] 2
## attr(,"method")
## [1] "tolNorm2"
```

```
## attr(,"useGrad")
## [1] FALSE
## attr(,"tol")
## [1] 7.549517e-15
```

```
## The input must be a numeric matrix, which means we have
## to covert df2 from a dataframe to a matrix
```

There are 3 variables in df2, but there are only 2 linearly independent variables since the generation of var2 is based on var1.

### Discussion II

TBD

### Discussion III

TBD